BA Insight

AI-Driven Predictive Customer Satisfaction
and Personalised Recommendations
for British Airways

Project Proposal

Build AI with AWS - May 2026
Week 3 Course Project, Option 2

Student	Aadit Aggarwal
Email	aadit.aggarwal01@sarasai.org
Institute	Saras AI Institute
Course	Build AI with AWS - May 2026
Assignment	Week 3 Course Project, Option 2
Project	BA Insight - British Airways customer-satisfaction AI
Date	5 November 2026
Version	1.0 - submitted for Week 3

One consistent project runs through Weeks 3, 4 and 5 of this course: this proposal (Week 3), the working build and GitHub repository with a summary (Week 4), and the final report with the Colab notebook, GitHub link and Streamlit deployment documentation (Week 5). Nothing in this proposal requires a paid AWS account or any payment.

1. Executive summary

British Airways (BA) receives thousands of free-text passenger reviews, yet the insight in them is slow to reach the people who can act on it. This proposal sets out BA Insight, an AI-driven system that reads BA passenger reviews, predicts how satisfied each passenger is, explains which service aspects drove that feeling, and turns the findings into specific, prioritised recommendations for BA teams.

The project follows the Week 3 Course Project, Option 2 brief exactly: it works on the British Airways reviews dataset supplied with the course, uses natural language processing (NLP) and sentiment analysis, builds a predictive model of customer satisfaction, extracts themes from negative feedback, and presents the results in a dashboard that BA staff can use. The whole system is designed around Amazon Web Services (AWS) - storage, serverless compute, managed AI services and a Streamlit front end - and will be built and run in Python in Google Colab, so every stage can be reproduced without a paid AWS account. Where a managed AWS service cannot be used directly in this environment, the project implements a local equivalent with the same interface and says so honestly.

By the end of Week 5 the project will deliver: a trained satisfaction-prediction model with measured accuracy; a topic and aspect analysis of what passengers praise and complain about; a working Streamlit application where a user can paste a review and receive a predicted satisfaction level, the aspects mentioned and a recommended action; and a public GitHub repository and Colab notebook documenting the complete implementation.

2. Problem statement

Airlines compete on experience as much as on price and schedule. A single delayed flight, a lost bag or a rude interaction can lose a customer for years, while consistently good service creates loyalty that advertising cannot buy. BA collects passenger feedback at scale, but free-text reviews are hard to use: they arrive in large volumes, they mix praise and complaint in the same review, and the themes that matter - seat comfort, food, staff service, entertainment, value for money, delays and baggage - are buried in unstructured prose.

The course dataset makes the problem concrete. The brief describes a dataset of over 2,000 British Airways reviews; the file actually supplied with the course (british_airways_reviews.csv.zip, Google Drive file 1d6m6567y2nQvLDL2T5GbMBK8ykVYbmlF) contains 1,300 reviews, each with a title and a review body and no numeric rating or structured service scores. This was verified on 5 November 2026 by downloading and inspecting the file in Colab: 1,300 rows, 3 columns (an index, title, reviews), and no missing values. That gap is itself part of the problem the project solves: because there is no labelled satisfaction score, the project must derive a defensible satisfaction signal from the text itself, validate it, and be transparent about it being a proxy rather than a ground-truth rating.

- The core problem: BA cannot currently predict, at the moment a review is written, whether that passenger is satisfied, neutral or dissatisfied, which aspects of the journey caused it, or what specific action would most improve the experience for similar passengers in future.

2.1 Why this matters

- Dissatisfied passengers rarely complain through one channel only; their reviews shape the choices of future travellers who read them.

- Service problems recur by route, cabin and season, but without aspect-level analysis BA cannot tell whether complaints are about delays, food, seats or staff.
- Generic responses ("we are sorry") do not fix root causes. Recommendations must be tied to evidence in the reviews.
- Manual reading does not scale to thousands of reviews, and simple star averages hide why satisfaction is rising or falling.

3. Objectives

The project has one primary objective and six supporting objectives, each measurable.

- PRIMARY: Build and evaluate a machine-learning model that predicts passenger satisfaction (Satisfied / Neutral / Dissatisfied) from review text, achieving at least 75% accuracy and a macro-F1 of at least 0.70 on a held-out test set, with results reported per class.
- Clean and prepare the full 1,300-review dataset: remove duplicates and empty reviews, normalise text, tokenise, remove stopwords and lemmatise, and document every decision and its effect on row counts.
- Perform sentiment analysis on every review and derive a transparent, reproducible satisfaction label from the text, validated by manual inspection of a sample of reviews.
- Identify the service aspects mentioned in each review (seat comfort, food and drink, staff service, in-flight entertainment, value for money, delays and punctuality, baggage, cleanliness, booking and check-in) and measure sentiment per aspect.
- Apply topic modelling to negative reviews to surface recurring complaint themes, and translate the top themes into specific, prioritised recommendations for BA.
- Build a working Streamlit application that accepts a new review, returns the predicted satisfaction, the aspects and sentiment detected, and a recommended action, with the model and pipeline saved so the app is reproducible.
- Document the full AWS-aligned architecture, the code, the results and the deployment steps in a public GitHub repository and an executed Colab notebook.

4. Target users and stakeholders

The system is designed for the people who can act on passenger feedback, and for the passengers whose experience it measures.

User / stakeholder	What they need	How BA Insight helps
BA customer-experience and service-recovery teams	To know, each week, which problems are growing and where	Aspect sentiment trends and the top complaint themes, with example reviews and counts
BA operations and route managers	To prioritise fixes by route, cabin and cause	Breakdowns by aspect (delays, baggage, seats) and recommendations ranked by frequency and severity
BA cabin crew and ground-staff trainers	To see what passengers praise or criticise about service	Staff-service sentiment and representative positive and negative comments for coaching

User / stakeholder	What they need	How BA Insight helps
BA product and catering teams	Evidence for seat, food and entertainment decisions	Aspect-level scores for seat comfort, food and drink and entertainment, tracked over time
Passengers (indirectly)	Faster, more relevant responses to problems	Recommendations that target the actual causes of dissatisfaction rather than generic apologies
Course assessors and future developers	A reproducible, well-documented implementation	Public GitHub repository, executed Colab notebook, and this proposal, summary and final report

5. Proposed solution and features

BA Insight is a text-analytics pipeline with a predictive model and a dashboard. A review enters the system, is cleaned and analysed, receives a predicted satisfaction level, is tagged with the service aspects it mentions and the sentiment towards each, and is stored with its prediction. Aggregated results feed a dashboard, and the recommendation engine converts the most frequent negative themes into actions.

F1 - Review intake and cleaning

Upload or paste reviews; de-duplicate, drop empty rows, strip HTML and URLs, normalise case and punctuation, and keep an audit trail of every cleaning decision.

F2 - Sentiment analysis and satisfaction labelling

VADER sentiment scoring (the local equivalent of Amazon Comprehend) gives a compound score per review; thresholds at +0.05 and -0.05 map it to Satisfied, Neutral or Dissatisfied. A manually checked sample validates the thresholds.

F3 - Satisfaction prediction

A TF-IDF (word and character n-grams) plus Logistic Regression classifier predicts the satisfaction label from review text. Random Forest, Linear SVC and XGBoost are compared on the same split so the choice is evidence-based, not assumed.

F4 - Aspect-based sentiment

Curated keyword lexicons for nine aspects (seat comfort; food and drink; staff service; in-flight entertainment; value for money; delays and punctuality; baggage; cleanliness; booking and check-in) tag each review and score sentiment per aspect.

F5 - Topic modelling of complaints

Latent Dirichlet Allocation (LDA) over negative reviews surfaces recurring themes (for example delays and compensation, lost or damaged baggage, unhelpful staff), each illustrated with representative reviews.

F6 - Personalised recommendations

A rule-based recommendation engine maps each (aspect, sentiment, topic) combination to a specific action - for example, proactive delay messaging and compensation guidance where delay complaints dominate - and ranks actions by frequency and severity.

F7 - Interactive dashboard (Streamlit)

Charts for satisfaction distribution, sentiment over the dataset, aspect scores and topic prevalence, plus a "try a review" panel that predicts satisfaction for new text and explains the aspects found.

F8 - Prediction API (serverless design)

A Lambda-style handler (lambda_handler.py) exposes predict(review_text) with the same input/output contract as the AWS API Gateway + Lambda design, so the app can move to AWS without changing its interface.

6. Data

Item	Detail
Source	Course-supplied British Airways reviews dataset, Google Drive file 1d6m6567y2nQvLDL2T5GbMBK8ykVYbmlF (british_airways_reviews.csv.zip)
Size (verified)	1,300 reviews. The brief states "over 2,000"; the supplied file contains 1,300. The project reports the verified figure and does not inflate it.
Columns	title (short review headline), reviews (free-text body). An index column is dropped. No numeric rating, date, route, cabin or service-score columns are present.
Missing values	None in either text column (verified).
Language	English, with informal spelling, abbreviations and occasional non-ASCII characters; cleaning handles these explicitly.
Labelling	No ground-truth satisfaction label exists. A label is derived from VADER sentiment (see F2) and treated as a proxy; model performance is therefore performance at predicting the sentiment-derived label, and this is stated wherever results are reported.
Ethics / privacy	Reviews are public passenger feedback supplied for coursework. No personal data is collected, no reviewer is identified, and no review text is published outside the coursework deliverables except as short illustrative quotations.

7. System architecture

The architecture follows the brief (cloud data pipeline, serverless APIs, dashboard) and is mirrored locally in Colab so it can be built and demonstrated without a paid AWS account. The data flow is:

- 1. INGEST - Reviews (CSV) are stored in an S3-style data lake (local data/ folder in Colab; Amazon S3 in the AWS design).
- 2. CLEAN - An AWS Lambda-style function (src/preprocess.py, also a Lambda handler) tokenises, removes stopwords, lemmatises and normalises text; cleaned data is versioned in the lake.

- 3. UNDERSTAND - Amazon Comprehend-style sentiment and key-phrase analysis (VADER + aspect lexicons locally) produces a sentiment score, an aspect tag set and a derived satisfaction label per review.
- 4. LEARN - Amazon SageMaker Autopilot-style model selection (scikit-learn comparison of Logistic Regression, Random Forest, Linear SVC and XGBoost) trains the satisfaction classifier; the best model and its metrics are saved as an artefact.
- 5. STORE - Predictions and aggregates are stored in a DynamoDB-style store (local SQLite/Parquet in Colab; Amazon DynamoDB in the AWS design).
- 6. SERVE - Amazon API Gateway + Lambda expose a prediction endpoint (lambda_handler.py), consumed by the Streamlit dashboard.
- 7. PRESENT - A Streamlit dashboard shows satisfaction distribution, aspect sentiment, topics and recommendations, and offers the single-review predictor.

This layered design keeps each concern separate: the NLP pipeline does not depend on the dashboard, the model is a saved artefact that any interface can load, and the AWS services can replace their local equivalents one at a time without rewriting the application logic.

8. AWS services and other technologies

The table below gives, for each technology, what it does in BA Insight and why it was chosen. Managed AWS services are the design target; the local equivalent is what runs in Colab, with the same interface, because no paid AWS account is used for this coursework.

Technology	Role in BA Insight	Why chosen / local equivalent
Amazon S3	Data lake: raw and cleaned review CSVs, model artefacts	Durable, versioned object storage; the standard place to land data in an AWS ML pipeline. Local: data/ folder.
AWS Lambda	Serverless cleaning and prediction functions	No server to manage; scales per request; matches the brief. Local: Python functions with the same handler signature.
Amazon Comprehend	Sentiment analysis, key phrases, language detection	Managed NLP with no model training needed. Local: VADER sentiment + aspect lexicons (no API key, reproducible offline).
Amazon SageMaker Autopilot	Automated model selection and training	Compares algorithms and preprocessing automatically. Local: scikit-learn model comparison over a fixed grid.
Amazon DynamoDB	Store for predictions and aggregates	Fast key-value lookups by review id. Local: SQLite / Parquet files with the same read/write interface.
Amazon API Gateway	HTTP endpoint in front of the prediction Lambda	Managed, secure API exposure. Local: the same handler is called directly by the Streamlit app.
Streamlit	Interactive dashboard and single-review predictor	Fastest way to turn a model into a usable app; named in the brief as a deployment option.
Python 3, pandas, scikit-learn	Data handling, TF-IDF, classification, LDA topic modelling	The standard, reproducible ML stack; runs in Google Colab.

Technology	Role in BA Insight	Why chosen / local equivalent
NLTK, VADER	Tokenisation, stopwords, lemmatisation, sentiment scoring	Lightweight, well-documented NLP; VADER is designed for short opinionated text.
Google Colab	Development and execution environment	Free GPU/CPU notebooks with the dataset available; the final notebook is the executable record of the work.
GitHub	Version control and public code hosting	Required by the course for Weeks 4-5; makes the work reviewable and reusable.

9. Implementation plan and timeline

Week	Milestone	Deliverable
Week 3	Proposal and infrastructure setup	This proposal: problem, solution, architecture, AWS services, timeline. Dataset verified (1,300 reviews). Repo created.
Week 4 (days 1-2)	Data pipeline	Cleaning, EDA, sentiment labelling, aspect tagging; charts and summary statistics.
Week 4 (days 3-4)	Modelling	TF-IDF + Logistic Regression baseline; comparison with Random Forest, Linear SVC, XGBoost; evaluation (accuracy, precision, recall, F1, confusion matrix).
Week 4 (day 5)	Topics and recommendations	LDA topics on negative reviews; recommendation engine; Week 4 summary PDF and GitHub update.
Week 5 (days 1-2)	Application	Streamlit app (dashboard + single-review predictor) and Lambda-style API handler; end-to-end testing.
Week 5 (days 3-4)	Final packaging	Executed Colab notebook shared for viewing; final PDF (GitHub link, Colab link, Streamlit deployment documentation, screenshots); submission and verification.

10. Methodology in brief

Text preprocessing

Lower-casing, URL/HTML removal, punctuation handling, tokenisation, English stopword removal, lemmatisation; title and body combined as the model input, with the title given as part of the text.

Sentiment and labelling

VADER compound score per review; Satisfied at >= +0.05, Dissatisfied at <= -0.05, Neutral between. Thresholds checked against 60 manually read reviews spanning the score range.

Features

TF-IDF over word unigrams/bigrams (max 5,000 features) plus character n-grams; aspect indicators as additional features in the final model.

Modelling

Stratified 80/20 train/test split with a fixed random seed; models compared on the same split; the best macro-F1 model is selected and re-evaluated with 5-fold cross-validation.

Evaluation

Accuracy, per-class precision/recall/F1, confusion matrix, and ROC-AUC (one-vs-rest). Errors are inspected qualitatively: which reviews the model gets wrong, and why.

Topics

LDA (8 topics) on dissatisfied reviews, with the top 10 words per topic and representative reviews; topics are named by a human, not by the algorithm.

11. Expected outcomes and success criteria

- A working, reproducible pipeline from raw CSV to trained model, with every step in the Colab notebook and the GitHub repository.
- A satisfaction classifier at or above 75% accuracy / 0.70 macro-F1 on the held-out test set (the target is stated in Section 3; if it is not reached, the report will say so and explain why).
- An aspect sentiment profile of BA as passengers describe it, and at least five evidence-based recommendations ranked by frequency and severity.
- A Streamlit app that a non-technical user can operate: paste a review, see the prediction and the reasons, and browse the dashboard.
- Clear documentation: proposal (Week 3), build summary (Week 4), and final report with repository, notebook and deployment documentation (Week 5).

12. Risks and mitigations

Risk	Likelihood / impact	Mitigation
No true satisfaction labels exist	High / High	Use a transparent VADER-derived proxy, validate it manually, and report results as prediction of that proxy; never present it as a ground-truth rating.
Class imbalance (very few neutral reviews)	Medium / Medium	Stratified splits, class weights, per-class metrics; report macro-F1, not accuracy alone.
Small dataset (1,300 reviews)	Medium / Medium	Keep the model simple (linear TF-IDF), use cross-validation, avoid large neural models that would overfit.
No paid AWS access	High / Low	Design for AWS, run local equivalents with identical interfaces in Colab; document the mapping in this proposal and the README.
Topic model produces vague topics	Medium / Low	Try 6, 8 and 10 topics, use coherence to choose, and name topics from representative reviews.
Streamlit deployment cannot be demonstrated live in the coursework window	Low / Low	Deployment is documented in full in the Week 5 final PDF (architecture, steps, access, configuration, maintenance) as instructed, without claiming a live deployment that does not exist.

13. Ethics, privacy and limitations

The dataset contains public passenger reviews supplied for coursework; no reviewer is contacted, identified or profiled, and no personal data beyond the review text is used. Sentiment-derived labels are a proxy for satisfaction, not a measurement of any individual passenger, and the project will not present them as such. The model learns from historical reviews and will reflect their biases - for example, passengers with bad experiences are more likely to write reviews - so the dashboard will describe the sample as self-selected reviewers, not all BA passengers. Finally, recommendations are suggestions for BA teams to consider, not operational decisions, and any action taken on them should be validated against BA operational data before it is implemented.

14. Sources

- Build AI with AWS (May 2026), Saras AI Institute - Week 3 Course Project, Option 2 brief: "AI-Driven Predictive Customer Satisfaction and Personalized Recommendations" (British Airways reviews dataset; AWS SageMaker Autopilot, Lambda, Streamlit).
- Course Project Introduction Document (Build AI with AWS): staged project timeline, Weeks 3-5.
- British Airways reviews dataset supplied with the course: british_airways_reviews.csv.zip, Google Drive file 1d6m6567y2nQvLDL2T5GbMBK8ykVYbmlF (verified 5 November 2026: 1,300 rows, columns title and reviews).
- Hutto, C. and Gilbert, E. (2014). VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text.

8. AWS services and other technologies

The table below gives, for each technology, what it does in BA Insight and why it was chosen. Managed AWS services are the design target; the local equivalent is what runs in Colab, with the same interface, because no paid AWS account is used for this coursework.

Technology	Role in BA Insight	Why chosen / local equivalent
Amazon S3	Data lake: raw and cleaned review CSVs, model artefacts	Durable, versioned object storage; the standard place to land data in an AWS ML pipeline. Local: data/ folder.
AWS Lambda	Serverless cleaning and prediction functions	No server to manage; scales per request; matches the brief. Local: Python functions with the same handler signature.
Amazon Comprehend	Sentiment analysis, key phrases, language detection	Managed NLP with no model training needed. Local: VADER sentiment + aspect lexicons (no API key, reproducible offline).
Amazon SageMaker Autopilot	Automated model selection and training	Compares algorithms and preprocessing automatically. Local: scikit-learn model comparison over a fixed grid.
Amazon DynamoDB	Store for predictions and aggregates	Fast key-value lookups by review id. Local: SQLite / Parquet files with the same read/write interface.
Amazon API Gateway	HTTP endpoint in front of the prediction Lambda	Managed, secure API exposure. Local: the same handler is called directly by the Streamlit app.
Streamlit	Interactive dashboard and single-review predictor	Fastest way to turn a model into a usable app; named in the brief as a deployment option.
Python 3, pandas, scikit-learn	Data handling, TF-IDF, classification, LDA topic modelling	The standard, reproducible ML stack; runs in Google Colab.
NLTK, VADER	Tokenisation, stopwords, lemmatisation, sentiment scoring	Lightweight, well-documented NLP; VADER is designed for short opinionated text.
Google Colab	Development and execution environment	Free GPU/CPU notebooks with the dataset available; the final notebook is the executable record of the work.
GitHub	Version control and public code hosting	Required by the course for Weeks 4-5; makes the work reviewable and reusable.

9. Implementation plan and timeline

Week	Milestone	Deliverable
Week 3	Proposal and infrastructure setup	This proposal: problem, solution, architecture, AWS services, timeline. Dataset verified (1,300 reviews). Repo created.
Week 4 (days 1-2)	Data pipeline	Cleaning, EDA, sentiment labelling, aspect tagging; charts and summary statistics.
Week 4 (days 3-4)	Modelling	TF-IDF + Logistic Regression baseline; comparison with Random Forest, Linear SVC, XGBoost; evaluation (accuracy, precision, recall, F1, confusion matrix).
Week 4 (day 5)	Topics and recommendations	LDA topics on negative reviews; recommendation engine; Week 4 summary PDF and GitHub update.
Week 5 (days 1-2)	Application	Streamlit app (dashboard + single-review predictor) and Lambda-style API handler; end-to-end testing.
Week 5 (days 3-4)	Final packaging	Executed Colab notebook shared for viewing; final PDF (GitHub link, Colab link, Streamlit deployment documentation, screenshots); submission and verification.

10. Methodology in brief

Text preprocessing

Lower-casing, URL/HTML removal, punctuation handling, tokenisation, English stopword removal, lemmatisation; title and body combined as the model input, with the title given as part of the text.

Sentiment and labelling

VADER compound score per review; Satisfied at >= +0.05, Dissatisfied at <= -0.05, Neutral between. Thresholds checked against 60 manually read reviews spanning the score range.

Features

TF-IDF over word unigrams/bigrams (max 5,000 features) plus character n-grams; aspect indicators as additional features in the final model.

Modelling

Stratified 80/20 train/test split with a fixed random seed; models compared on the same split; the best macro-F1 model is selected and re-evaluated with 5-fold cross-validation.

Evaluation

Accuracy, per-class precision/recall/F1, confusion matrix, and ROC-AUC (one-vs-rest). Errors are inspected qualitatively: which reviews the model gets wrong, and why.

Topics

LDA (8 topics) on dissatisfied reviews, with the top 10 words per topic and representative reviews; topics are named by a human, not by the algorithm.

11. Expected outcomes and success criteria

- A working, reproducible pipeline from raw CSV to trained model, with every step in the Colab notebook and the GitHub repository.
- A satisfaction classifier at or above 75% accuracy / 0.70 macro-F1 on the held-out test set (the target is stated in Section 3; if it is not reached, the report will say so and explain why).
- An aspect sentiment profile of BA as passengers describe it, and at least five evidence-based recommendations ranked by frequency and severity.
- A Streamlit app that a non-technical user can operate: paste a review, see the prediction and the reasons, and browse the dashboard.
- Clear documentation: proposal (Week 3), build summary (Week 4), and final report with repository, notebook and deployment documentation (Week 5).

12. Risks and mitigations

Risk	Likelihood / impact	Mitigation
No true satisfaction labels exist	High / High	Use a transparent VADER-derived proxy, validate it manually, and report results as prediction of that proxy; never present it as a ground-truth rating.
Class imbalance (very few neutral reviews)	Medium / Medium	Stratified splits, class weights, per-class metrics; report macro-F1, not accuracy alone.
Small dataset (1,300 reviews)	Medium / Medium	Keep the model simple (linear TF-IDF), use cross-validation, avoid large neural models that would overfit.
No paid AWS access	High / Low	Design for AWS, run local equivalents with identical interfaces in Colab; document the mapping in this proposal and the README.
Topic model produces vague topics	Medium / Low	Try 6, 8 and 10 topics, use coherence to choose, and name topics from representative reviews.
Streamlit deployment cannot be demonstrated live in the coursework window	Low / Low	Deployment is documented in full in the Week 5 final PDF (architecture, steps, access, configuration, maintenance) as instructed, without claiming a live deployment that does not exist.

13. Ethics, privacy and limitations

The dataset contains public passenger reviews supplied for coursework; no reviewer is contacted, identified or profiled, and no personal data beyond the review text is used. Sentiment-derived labels are a proxy for satisfaction, not a measurement of any individual passenger, and the project will not present them as such. The model learns from historical reviews and will reflect their biases - for example, passengers with bad experiences are more likely to write reviews - so the dashboard will describe the sample as self-selected reviewers, not all BA passengers. Finally, recommendations are suggestions for BA teams to consider, not operational decisions, and any action taken on them should be validated against BA operational data before it is implemented.

14. Sources

- Build AI with AWS (May 2026), Saras AI Institute - Week 3 Course Project, Option 2 brief: "AI-Driven Predictive Customer Satisfaction and Personalized Recommendations" (British Airways reviews dataset; AWS SageMaker Autopilot, Lambda, Streamlit).
- Course Project Introduction Document (Build AI with AWS): staged project timeline, Weeks 3-5.
- British Airways reviews dataset supplied with the course: british_airways_reviews.csv.zip, Google Drive file 1d6m6567y2nQvLDL2T5GbMBK8ykVYbmlF (verified 5 November 2026: 1,300 rows, columns title and reviews).
- Hutto, C. and Gilbert, E. (2014). VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text.